

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN - ĐHQG-HCM

KHOA CÔNG NGHỆ PHẦN MỀM

MÔN HỌC: SE373 · KỸ THUẬT XÂY DỰNG HỆ THỐNG AGENTIC AI

BÁO CÁO BÀI TẬP VỀ NHÀ #03 (BTVN#3)

ĐỀ TÀI: DỰNG AGENT ĐẶT VÉ MÁY BAY BẰNG LANGCHAIN VÀ XÂY DỰNG LỚP HARNESS

1. TỔNG QUAN HỆ THỐNG VÀ KIẾN TRÚC TỔNG THỂ

Theo bài giảng **Buổi 03: Agent Fundamentals**, công thức cốt lõi của một hệ thống Agentic AI là:

Agent = Goal + Tools + Loop + Termination

Hệ thống được phân định rõ ràng giữa **Model** (LLM - đề xuất bước suy luận và lời gọi tool lúc chạy) và **Harness** (Code kiểm soát của kỹ sư phần mềm - dựng ngữ cảnh, kiểm quyền, thực thi tool, ghi state, và xét điều kiện dừng).

Toàn bộ mã nguồn bài tập gồm các file thành phần:

- * [mock_tools.py](#) : Mock database và các công cụ đặt vé máy bay theo chuẩn Structured JSON.
- * [harness.py](#) : Lớp kiểm soát Harness với đầy đủ 4 lớp yêu cầu.
- * [agent_react.py](#) : Agent triển khai theo mẫu ReAct (Reasoning + Acting).
- * [agent_plan_execute.py](#) : Agent triển khai theo mẫu Plan-then-Execute.
- * [agent_hybrid.py](#) : Agent triển khai theo Mẫu Lai (Plan + ReAct / Re-planning).
- * [evaluate.py](#) : Bộ benchmark 4 kịch bản đánh giá hiệu quả định lượng và định tính.
- * [main.py](#) : Giao diện CLI chạy demo và tương tác trực tiếp.

2. THIẾT KẾ CÁC TOOL MOCKUP (SLIDE 12, 13, 56, 66)

Để tránh **Failure Mode 4 ("Tin vào dữ liệu sai")** và tình trạng Agent diễn dịch mơ hồ chuỗi văn bản rỗng, toàn bộ công cụ được thiết kế trả về **Structured Output (JSON)** có định danh và mã lỗi rõ ràng:

1. `search_flights(origin, destination, date)` :
2. Kiểm tra định dạng ngày `YYYY-MM-DD` (Slide 56). Nếu sai định dạng, trả mã `INVALID_DATE_FORMAT` kèm `hint` hướng dẫn.
3. Trả về danh sách chuyến bay gồm: `flight_id` , `airline` , `depart_time` , `price` , `seats_count` , `refundable` .
4. Nếu không có chuyến bay, trả về `{"status": "empty", "flights": []}` thay vì chuỗi rỗng.
5. `check_seat(flight_id)` :
6. Tra cứu chi tiết ghế trống (`available_seats`), giá vé chính xác, trạng thái hoàn vé (`refundable`).
7. Hỗ trợ phát hiện tình trạng hết vé (`sold_out`).
8. `book_seat(flight_id, seat_number, passenger_name)` :
9. Giữ chỗ tạm thời, đưa trạng thái booking về `held` , sinh mã định danh duy nhất (ví dụ: `BK-VN122-101`), khóa ghế trong DB.
10. `pay(booking_code, payment_method)` :
11. Thanh toán tiền vé cho mã đặt chỗ, chuyển trạng thái sang `confirmed` và `paid = True` .
12. `get_booking(booking_code)` :
13. Dùng để phục vụ cơ chế kiểm chứng chéo độc lập (Slide 44).

3. CÀI ĐẶT CÁC LỚP HARNESS (YÊU CẦU 1)

Tuân thủ sơ đồ quyết định 5 điều kiện dừng (Slide 34) và checklist vòng lặp (Slide 35), lớp `AgentHarness` trong `harness.py` cài đặt trọn vẹn 4 lớp:

3.1. Lớp 1: Ràng buộc là dữ liệu (Data Constraints - Slide 63)

- **Ý tưởng:** Không để ràng buộc lười xa và bị quên lãng trong chuỗi chat dài của LLM (khắc phục **Failure Mode 3: "Quên yêu cầu ban đầu"**).
- **Cài đặt:** Lớp `FlightConstraints` lưu cấu hình:

```
python constraints = FlightConstraints( origin="SGN", destination="DAD", date="2026-10-07",
depart_before="12:00", max_price=2_000_000 )
```

Hàm `is_flight_valid()` được gọi để kiểm tra khách quan giá tiền và giờ bay trước khi chấp thuận.

3.2. Lớp 2: Kiểm quyền trước khi gọi tool (Permission / Human Approval - Slide 35, 41)

- **Thời điểm chạy:** Checklist #0 - Chạy **TRƯỚC** khi thực thi tool nhạy cảm.
- **Quy tắc:**

- Khi gọi `book_seat` : Nếu giá vé > 1.500.000đ (vượt hạn mức tự chủ) HOẶC vé thuộc loại không được hoàn trả (`refundable == False`), Harness chặn lại.
- Khi gọi `pay` : Thao tác tài chính luôn yêu cầu xác nhận.
- Khi bị chặn, Harness không cho tool thực thi và chuyển trạng thái sang `Cần con người (Approval)` , đồng thời kích hoạt bàn giao.

3.3. Lớp 3: Tiêu chí hoàn thành kiểm bằng code (Computational Sensor - Slide 36, 43, 44)

- **Thời điểm chạy:** Checklist #1 - Chạy **SAU** khi nhận Observation.
- **Nguyên tắc:** Tuyệt đối không phụ thuộc vào việc model tự tuyên bố: "Tôi đã đặt vé xong".
- **Cài đặt:** Lớp `ComputationalSensor` kiểm chứng bằng biểu thức logic khách quan:

```
python get_booking(code).status == "confirmed" and paid == True \ and price <= 2_000_000 \
and depart_date == "2026-10-07" and depart_time < "12:00"
```

 Nếu toàn bộ vị từ thỏa mãn, Harness ghi nhận kết thúc dạng `Đạt mục tiêu (Success)` bình thường.

3.4. Lớp 4: Bàn giao cho con người (Handoff Report - Slide 48)

- **Nguyên tắc:** "Bàn giao tốt là bàn giao mà người nhận trả lời được trong 30 giây."
- **Cấu trúc báo cáo:**
- *Trạng thái hiện tại:* Đã thực hiện tới bước nào, tác dụng phụ đã tạo (mã vé đang hold, tiền đã trừ chưa).
- *Những gì đã thử:* Tóm tắt 3-4 hành động gần nhất và kết quả.
- *Câu hỏi cụ thể:* Đặt câu hỏi đúng trọng tâm (ví dụ: "Chuyến VN122 có giá 1.850.000đ vượt hạn mức 1.500.000đ và không hoàn vé. Bạn có duyệt đặt ghế 12A không?").

3.5. Bổ sung: Bộ phát hiện lặp và Ngân sách cứng (Slide 15, 46)

- **LoopDetector** : Cài đặt bằng `deque(maxlen=6)` . Tạo fingerprint `(tool, sorted(args))` . Nếu lặp lại ≥ 2 lần hoặc đại lượng tiến triển đứng yên suốt 4 vòng → Dừng với lỗi `LOOP` hoặc `STALL` .
- **Hard Limits:** Giới hạn cứng `max_turns = 8` và timeout `30s` để ngăn chặn tốn token theo cấp số nhân (Slide 14).

4. CÀI ĐẶT AGENT VỚI 3 MẪU THIẾT KẾ (YÊU CẦU 2)

4.1. Mẫu 1: ReAct (Reasoning + Acting - Slide 18–21)

- **Cơ chế:** Vòng lặp liên tục: `Thought → Action → Observation → Thought ...`
- **Ưu điểm:** Dữ kiện từ môi trường đi vào giữa chuỗi suy luận; hướng đi được điều chỉnh linh hoạt sau từng observation.
- **Rủi ro:** Nguy cơ lặp vô hạn hoặc trôi mục tiêu nếu không có `LoopDetector` và `Constraints` của Harness.

4.2. Mẫu 2: Plan-then-Execute (Slide 22–23)

- **Cơ chế:** Gọi LLM **một lần duy nhất** để sinh trọn vẹn danh sách các bước (`Plan` JSON). Sau đó bộ thực thi duyệt tuần tự qua từng bước theo mô hình thác nước (Waterfall).
- **Ưu điểm:** Kế hoạch nhìn thấy được toàn bộ trước khi chạy, dễ kiểm duyệt trước và ước lượng chi phí/token chính xác.
- **Nhược điểm (Đánh đổi): Kém linh hoạt, dễ vỡ.** Nếu một bước ở giữa gặp sự cố (ví dụ: chuyển bay dự định bị hết vé), toàn bộ kế hoạch phía sau bị vô hiệu hóa vì không có cơ chế thích nghi.

4.3. Mẫu 3: Mẫu Lai (ReAct + Plan / Dynamic Re-planning - Slide 24–26)

- **Cơ chế:**
 - Ban đầu lập kế hoạch tổng thể.
 - Thực thi từng bước thông qua Harness.
 - Sau mỗi bước, kiểm tra điều kiện: "*Observation đối đáng kể?*" (nhận diện qua status: `sold_out` , `seat_unavailable` , `error`).
 - Nếu có biến động → Kích hoạt bộ **Re-planner** để sinh lại kế hoạch mới dựa trên dữ kiện vừa quan sát được.
- **Ưu thế:** Kết hợp ưu điểm định hướng dài hạn của Plan với khả năng thích ứng động của ReAct.

5. THỰC NGHIỆM VÀ ĐÁNH GIÁ SO SÁNH HIỆU QUẢ (YÊU CẦU 3)

Chương trình `evaluate.py` đã thực thi 4 kịch bản benchmark thực tế với mô hình **Google Gemini** (`gemini-3.5-flash-lite`) kết hợp lớp kiểm soát `AgentHarness` :

5.1. Bảng tổng hợp kết quả Benchmark thực nghiệm thực tế

Kịch bản thử nghiệm	Mẫu Agent	Trạng thái	Số vòng (Turns)	Thời gian (s)	Loại dừng ghi nhận bởi Harness
Kịch bản 1: Môi trường thuận lợi <i>(Happy Path)</i>	ReAct	✓ THÀNH CÔNG	4	4.77s	Đạt mục tiêu (Booking BK-VJ604-101 hợp lệ, 1.450.000đ)
	Plan-then-Execute	✓ THÀNH CÔNG	4	1.55s	Đạt mục tiêu (Booking BK-VN122-101 hợp lệ, 1.850.000đ)
	Mẫu Lai (Hybrid)	✓ THÀNH CÔNG	4	1.62s	Đạt mục tiêu (Booking BK-VN122-101 hợp lệ, 1.850.000đ)
Kịch bản 2: Biến động môi trường <i>(Chuyển VN122 hết chỗ)</i>	ReAct	✓ THÀNH CÔNG	4	6.29s	Đạt mục tiêu (Thích ứng đổi sang VJ604: 1.450.000đ)
	Plan-then-Execute	✗ THẤT BẠI	2	1.59s	Kế hoạch thất bại (Plan Stale - Kế hoạch cứng bị vỡ)
	Mẫu Lai (Hybrid)	✓ THÀNH CÔNG	5	2.90s	Đạt mục tiêu (Re-plan thích ứng sang VJ604)
Kịch bản 3: Kích hoạt kiểm quyền <i>(Yêu cầu người duyệt)</i>	ReAct	⏸ DỪNG AN TOÀN	3	2.10s	Cần con người (Approval - In Handoff 30s)
	Plan-then-Execute	⏸ DỪNG AN TOÀN	3	1.64s	Cần con người (Approval - In Handoff 30s)
	Mẫu Lai (Hybrid)	⏸ DỪNG AN TOÀN	3	1.50s	Cần con người (Approval - In Handoff 30s)
Kịch bản 4: Lỗi mạng lặp lại <i>(Loop / Timeout Detection)</i>	ReAct	● BẮT LỖI LẶP	2	3.09s	Phát hiện lặp (LoopDetector ngắt luồng)
	Plan-then-Execute	✗ THẤT BẠI	1	1.62s	Kế hoạch thất bại (Plan Stale)

Kịch bản thử nghiệm	Mẫu Agent	Trạng thái	Số vòng (Turns)	Thời gian (s)	Loại dừng ghi nhận bởi Harness
	Mẫu Lai (Hybrid)	BẮT LỖ LẶP	2	2.88s	Phát hiện lặp (LoopDetector ngắt luồng)

5.2. Phân tích chi tiết kết quả từng kịch bản

Kịch bản 1 (Môi trường thuận lợi):

- Cả 3 mẫu thiết kế đều hoàn thành xuất sắc trong 4 turns (Search → Check → Book → Pay).
- Plan-then-Execute (1.55s)** và **Mẫu Lai (1.62s)** chạy nhanh gấp **gần 3 lần** so với **ReAct (4.77s)** vì không phải gọi LLM suy luận lại ở mỗi vòng lặp.

Kịch bản 2 (Biến động môi trường - Chuyển VN122 hết chỗ):

- Plan-then-Execute thất bại hoàn toàn:** Do kế hoạch được lập cứng từ đầu, khi bước `check_seat` phát hiện VN122 hết chỗ, toàn bộ chuỗi thác nước phía sau sụp đổ. Đây là minh chứng rõ nét cho nhận định tại **Slide 23**: "Lỗi ở bước đầu sẽ làm hỏng toàn bộ sau".
- ReAct và Mẫu Lai thành công xuất sắc:**
- `ReAct` nhận observation `sold_out`, lập tức suy luận chọn chuyển thay thế `VJ604` (1.450.000đ).
- `Mẫu Lai` nhận thấy "Observation đối đáng kể", kích hoạt cơ chế **Re-plan**, cập nhật lại lộ trình và đặt thành công vé thay thế.

Kịch bản 3 (Kiểm quyền trước khi thực thi - Human Approval):

- Khi tắt chế độ `auto_approve`, ngay tại bước `book_seat` (do giá vé 1.850.000đ vượt hạn mức tự chủ 1.500.000đ và vé không hoàn trả), lớp Harness ở Checklist #0 đã chặn lại trước khi gọi tool.
- Hệ thống xuất ra **Báo cáo Bàn giao Handoff 30 giây (Slide 48)** với đầy đủ 3 thành phần:
- Trạng thái hiện tại:* Vòng 3/8, chưa tạo booking hay phát sinh trừ tiền.
- Những gì đã thử:* Đã tìm chuyến bay và kiểm tra chỗ của VN122.
- Câu hỏi cụ thể:* "Chuyến VN122 có Giá 1.850.000đ vượt hạn mức tự chủ 1.500.000đ và Vé loại không được hoàn trả. Bạn có duyệt đặt ghế 12A không?".

Kịch bản 4 (Phát hiện lặp và lỗi mạng):

- Khi mock tool giả lập lỗi mạng `TIMEOUT`, Agent cố gắng gọi lại tool tìm kiếm.
- `LoopDetector` của Harness đối chiếu fingerprint (`tool, args`) trong hàng đợi trượt `deque` và **ngắt luồng ngay lập tức ở vòng 2**, ngăn chặn Agent rơi vào vòng lặp vô hạn và bảo vệ ngân sách token (Slide 14, 54).

5.3. Bảng so sánh tổng hợp 3 mẫu thiết kế (Theo tiêu chuẩn Slide 26)

Tiêu chí so sánh	ReAct	Plan-then-Execute	Mẫu Lai (Hybrid)
Số lần gọi LLM	Nhiều (mỗi vòng 1 lần)	Ít nhất (1 lần ban đầu)	Vừa phải (chỉ gọi lại khi Re-plan)
Thời gian chạy (Happy Path)	~4.77s	~1.55s (Nhanh nhất)	~1.62s
Khả năng thích ứng biến động	Rất cao	Kém (Dễ vỡ kế hoạch)	Rất cao (Có Re-plan)
Kiểm duyệt an toàn (Harness)	Tuân thủ tuyệt đối	Tuân thủ tuyệt đối	Tuân thủ tuyệt đối
Rủi ro chính (Slide 26)	Lập vô hạn / Trôi mục tiêu	Kế hoạch lỗi thời	Độ phức tạp mã nguồn
Khi nào nên chọn (Slide 26)	Không đoán được số bước	Cần duyệt trước kế hoạch & chi phí	Tác vụ dài, môi trường biến động

6. KẾT LUẬN VÀ BÀI HỌC KINH NGHIỆM

- 1. Ranh giới Model và Harness:
- 2. Model chỉ nên chịu trách nhiệm đề xuất bước tiếp theo lúc chạy (Step 02 trong Slide 10).
- 3. 4 bước còn lại (Dựng ngữ cảnh, Gọi tool an toàn, Ghi state, Xét điều kiện dừng) bắt buộc phải do lớp **Harness** bằng code kiểm soát độc lập.
- 4. Quy tắc chọn mẫu thiết kế (Slide 26):
- 5. Dùng **Plan-then-Execute** cho các tác vụ tĩnh, quy trình rõ ràng, cần kiểm duyệt ngân sách chặt chẽ từ đầu.
- 6. Dùng **ReAct** cho tác vụ ngắn, khó đoán trước số bước.
- 7. Dùng **Mẫu Lai (ReAct + Plan)** cho các tác vụ dài hơi hoạt động trong môi trường biến động cao.
- 8. Tầm quan trọng của Sensor Computational:
- 9. Không bao giờ được dùng LLM để tự đánh giá tác vụ của chính nó nếu có thể đo bằng code (HTTP status, DB query, regex, schema validation). Sensor computational chạy trong vài mili-giây, không tốn token và xác định 100%.